md_to_docx Capability Showcase

Comprehensive Feature Demonstration and Documentation

Technical Demonstration

Version: 2.0 | **Classification:** Internal Use Only

Document Revision History

Version	Date	Author	Changes
2.0	2026-03-14	Your Name	Added cross-references, standalone images, code examples, and revision tracking
1.1	2026-03-10	Your Name	Updated content structure and added table formatting examples
1.0	2026-03-01	Your Name	Initial release with basic document structure and formatting

Table of Contents

Table of Contents 2

1. Introduction 1

  1.1 Purpose and Scope 1

2. Document Structure 2

  2.1 Heading Hierarchy 2

    2.1.1 Level 3 Test2 2

  2.2 Heading Customization 2

3. Typography and Text Formatting 3

  3.1 Basic Text Styles Tatia 3

  3.2 Inline Code 3

  3.3 Blockquotes 3

4. Lists and Enumeration 4

  4.1 Unordered Lists 4

  4.2 Ordered Lists 4

  4.3 Nested Lists 4

5. Tables 5

  5.1 Standard Data Table 5

  5.2 Table Without Header 5

  5.3 Wide Table with Text Wrapping 5

  5.4 Table with Custom Column Widths 6

6. Images and Figures 7

  6.1 Standalone Images 7

    6.1.1 Full-Width Image 7

    6.1.2 Left-Aligned Image 7

    6.1.3 Centered Medium Image 7

  6.2 Figure Groups 8

7. Cross-References and Linking 9

  7.1 Internal References 9

  7.2 External Hyperlinks 9

8. Code Examples 10
8.1 Python Code Block 10
8.2 JavaScript Code Block 10
9. Special Elements 11
9.1 Alert Boxes 11
9.2 Horizontal Rules 11
9.3 Vertical Space 13
Appendix 14
A. Supported Markdown Extensions 14
A.1 GitHub Flavored Markdown 14
9.3.1 A.1.1 Table Alignment 14
9.3.2 A.2 Custom Extensions 14
B. Configuration Options 14
B.1 Document Metadata 14
B.1.1 Dynamic Fields 14
B.2 Styling Options 15
C. Property Substitution 15
C.1 Usage Examples 15
C.2 Configuration Data 15
10. Introduction 16
10.1 Purpose and Scope 16
11. Document Structure 17
11.1 Heading Hierarchy 17
11.1.1 Level 3 Heading 17
11.2 Heading Customization 17
12. Typography and Text Formatting 18
12.1 Basic Text Styles 18
12.2 Inline Code 18
12.3 Blockquotes 18
13. Lists and Enumeration 19
13.1 Unordered Lists 19
13.2 Ordered Lists 19

13.3 Nested Lists 19
14. Tables 20
14.1 Standard Data Table 20
14.2 Table Without Header 20
14.3 Wide Table with Text Wrapping 20
14.4 Table with Custom Column Widths 21
15. Images and Figures 22
15.1 Standalone Images 22
15.1.1 Full-Width Image 22
15.1.2 Left-Aligned Image 22
15.1.3 Centered Medium Image 22
15.2 Figure Groups 23
16. Cross-References and Linking 24
16.1 Internal References 24
16.2 External Hyperlinks 24
17. Code Examples 25
17.1 Python Code Block 25
17.2 JavaScript Code Block 25
18. Special Elements 26
18.1 Alert Boxes 26
18.2 Horizontal Rules 26
18.3 Vertical Space 28
Appendix 29
A. Supported Markdown Extensions 29
A.1 GitHub Flavored Markdown 29
18.3.1 A.1.1 Table Alignment 29
18.3.2 A.2 Custom Extensions 29
B. Configuration Options 29
B.1 Document Metadata 29
B.1.1 Dynamic Fields 29
B.2 Styling Options 30
C. Property Substitution 30

C.1 Usage Examples 30
C.2 Configuration Data 30

1. Introduction

This document serves as a comprehensive demonstration of the md_to_docx converter capabilities. It showcases the various formatting features and document elements that can be automatically converted from Markdown syntax to professional Microsoft Word documents.

1.1 Purpose and Scope

The primary objectives of this showcase are to:

1. **Demonstrate document structure capabilities** including multi-level headings, automatic table of contents generation, and section organization
2. **Present text formatting features** such as emphasis, inline code, and paragraph styling
3. **Illustrate list functionality** including ordered, unordered, and nested list structures
4. **Exhibit table generation** with various formatting options and column width controls
5. **Showcase image handling** including standalone images, figure groups, and size controls
6. **Highlight cross-referencing** capabilities for figures, tables, and sections

2. Document Structure

2.1 Heading Hierarchy

The converter supports six levels of headings, automatically numbered and styled with a professional color scheme:

2.1.1 Level 3 Test2

This demonstrates the third level of the document hierarchy.

2.1.1.1 Level 4 Heading

Fourth level headings provide additional granularity for complex documents.

2.1.1.1.1 Level 5 Heading

Fifth level headings for detailed subsections.

2.1.1.1.1.1 Level 6 Heading

The deepest level of heading hierarchy available.

2.2 Heading Customization

Headings can be excluded from the table of contents using the `{.notoc}` attribute, or numbering can be suppressed with `{.nonumber}`.

3. Typography and Text Formatting

3.1 Basic Text Styles Tatia

The converter supports standard Markdown text formatting:

Italic text is used for emphasis or citations.

Bold text draws attention to important information.

Bold and italic combined creates strong emphasis.

~~Strikethrough~~ indicates removed or deprecated content.

3.2 Inline Code

Technical terms and code snippets can be displayed inline using backticks: `function_name()`, `variable_value`, or `CSS_property`.

3.3 Blockquotes

> *This is a standard blockquote. It can be used for important notes, citations, or highlighted information that stands out from the main text flow.*

Blockquotes feature a distinctive left border and italicized text for visual differentiation.

4. Lists and Enumeration

4.1 Unordered Lists

Simple bullet points for itemized information:

- First item in the list
- Second item with additional context
- Third item demonstrating the format

4.2 Ordered Lists

Numbered sequences for procedural documentation:

1. First step in the process
2. Second step with specific details
3. Third step completing the sequence

4.3 Nested Lists

Lists can be nested to show hierarchical relationships:

1. **Main category A**
    1. Sub-item A.1
    2. Sub-item A.2
        1. Detail A.2.1
        2. Detail A.2.2
    3. Sub-item A.3
2. **Main category B**
    1. Sub-item B.1
    2. Sub-item B.2
3. **Main category C**
    1. Sub-item C.1

This demonstrates the converter's ability to handle complex list structures with proper indentation and formatting.

5. Tables

5.1 Standard Data Table

Tables support headers, alternating row shading, and full-width formatting:

Feature	Status	Priority	Notes
Heading Numbering	Implemented	High	Configurable via YAML
Cross-references	Implemented	High	Bold + Italic style
Image Resizing	Implemented	Medium	Size classes supported
Table Styling	Implemented	High	Auto-formatting applied
Code Blocks	Implemented	Medium	Syntax highlighting

Table 4: *Feature implementation status and priorities.*

As shown in Table 4, all major features have been successfully implemented with appropriate priority levels assigned.

5.2 Table Without Header

Simple tables without headers are also supported:

Row 1, Cell 1	Row 1, Cell 2	Row 1, Cell 3	Row 1, Cell 4
Row 2, Cell 1	Row 2, Cell 2	Row 2, Cell 3	Row 2, Cell 4
Row 3, Cell 1	Row 3, Cell 2	Row 3, Cell 3	Row 3, Cell 4

Table 3: *Data visualization example without headers.*

5.3 Wide Table with Text Wrapping

Tables automatically handle text wrapping for content that exceeds column widths:

Component	Description	Implementation Details	Status
Parser	Marko library with GFM extensions	Handles all standard Markdown plus GitHub-style alerts and tables	Active

Builder	Document assembly engine	Manages section breaks, page numbering, and TOC generation	Active
Styles	Custom style definitions	Heading hierarchy, code blocks, captions, and alerts	Active
Images	Processing pipeline	Size calculation, aspect ratio preservation, placement	Active

Table 5: *System architecture components with detailed descriptions.*

Refer to Table 5 for a complete overview of the system architecture.

5.4 Table with Custom Column Widths

Tables can have custom column widths specified as percentages using the `{col-widths="..."}` directive placed immediately after the table:

Short	Medium	Long Description
ID	Category	Detailed explanation of the feature or component
001	Parser	Handles Markdown syntax parsing with GFM extensions
002	Builder	Assembles the document structure and applies formatting

Table 6: *Example table with custom column width distribution.*

In this example, the first column occupies 15% of the table width, the second column 25%, and the third column (containing longer descriptions) occupies 60% of the available space.

6. Images and Figures

6.1 Standalone Images

6.1.1 Full-Width Image

Large images can be displayed at full content width for maximum impact:

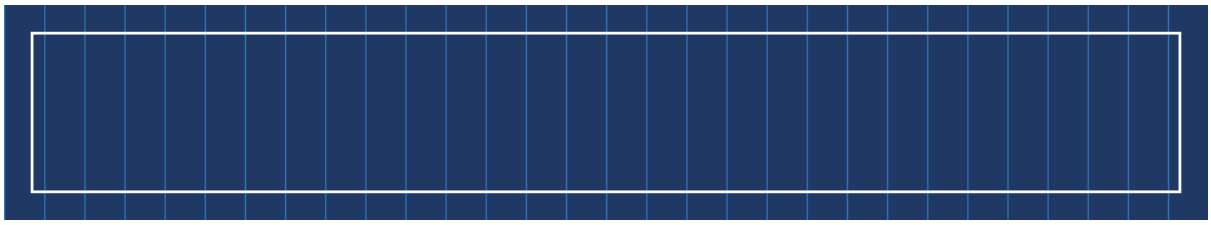


Figure 4: *Complete system overview diagram showing all major components and their relationships.*

As illustrated in Figure 4, the system architecture consists of four primary modules working in concert.

6.1.2 Left-Aligned Image

Images can be positioned on the left side with text flowing alongside:

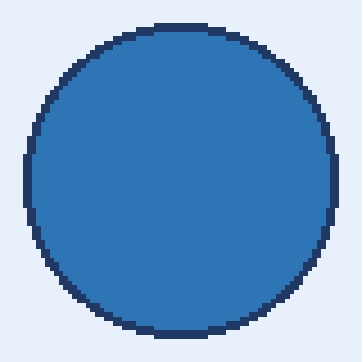


Figure 1: *Individual module icon representing the parsing engine.*

This layout demonstrates how content wraps around left-aligned images. The parsing module serves as the foundation of the conversion pipeline, responsible for transforming raw Markdown syntax into an abstract syntax tree (AST) that the builder can process.

6.1.3 Centered Medium Image

The default presentation centers images within the content area:

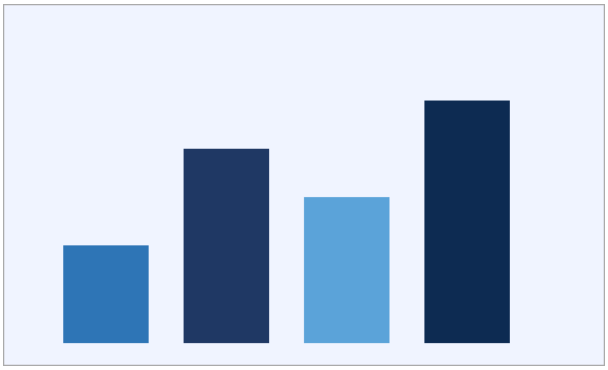
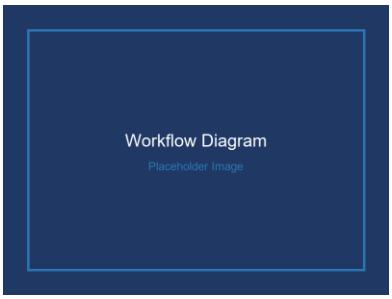


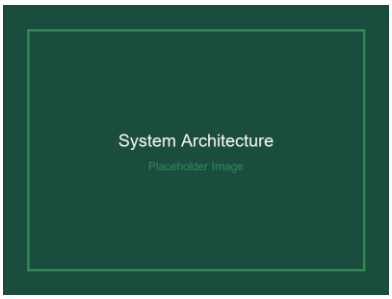
Figure 5: *Data flow diagram showing information pathways through the system.*

6.2 Figure Groups

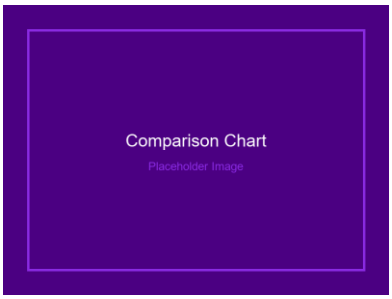
Multiple images can be displayed side-by-side for comparison purposes:



a) Input Processing



b) Output Generation



c) Quality Control

Figure 6: *Three-phase processing pipeline: input parsing, document building, and quality validation.*

The figure group above, referenced as Figure 6, demonstrates how multiple related images can be presented together with individual sub-captions (a, b, c) and an overall group caption.

7. Cross-References and Linking

7.1 Internal References

The document supports cross-referencing to:

Sections (e.g., Section 4.2 covers list functionality)

7.2 External Hyperlinks

External resources can be linked directly: Markdown Specification or Microsoft Word Documentation.

8. Code Examples

8.1 Python Code Block

```python
def process_markdown(source_file, config):
    """
    Main entry point for markdown processing.

    Args:
        source_file: Path to markdown input
        config: Dictionary of configuration options

    Returns:
        Document: Generated Word document object
    """
    builder = DocumentBuilder(config)
    builder.setup()

    with open(source_file, 'r') as f:
        content = f.read()

    builder.add_content(content)
    return builder.doc
```

Code Block 1: Python implementation of the main processing function.

8.2 JavaScript Code Block

```javascript
// Configuration parser for document settings
function parseConfig(configPath) {
  const fs = require('fs');
  const yaml = require('js-yaml');

  try {
    const config = yaml.load(fs.readFileSync(configPath, 'utf8'));
    return {
      margins: config.page || {},
      header: config.header || {},
      footer: config.footer || {},
      document: config.document || {}
    };
  } catch (error) {
    console.error('Configuration error:', error);
    return {};
  }
}
```

Code Block 2: JavaScript configuration parser implementation.

9. Special Elements

9.1 Alert Boxes

GitHub-style alerts provide visual emphasis for different message types:

> **[NOTE]** This is an informational note. It provides additional context or helpful information that supplements the main content.

> **[TIP]** This is a helpful tip. Tips offer practical advice or shortcuts that can improve workflow efficiency.

> **[WARNING]** This is a warning message. Warnings indicate potential issues or important considerations that require attention.

> **[CAUTION]** This is a caution alert. Cautions highlight actions that could cause problems if not handled carefully.

9.2 Horizontal Rules

Horizontal rules can be used to separate sections visually:

They create a clear break without starting a new page.

9.3 Vertical Space

The `:::space` directive provides precise vertical spacing control, ideal for fine-tuning layout on cover pages and between sections.

Use `lines` to insert space measured in body-text lines (11 pt per line):

Or use `pt` for an exact point value:

Both examples above create white space without forcing a page break.

Appendix

A. Supported Markdown Extensions

A.1 GitHub Flavored Markdown

The converter fully supports GFM syntax including:

- Tables with alignment
- Task lists (converted to bullet points)
- Strikethrough text
- Autolinks

9.3.1 A.1.1 Table Alignment

Tables support column alignment through colon placement:

Left	Center	Right
L1	C1	R1
L2	C2	R2

9.3.2 A.2 Custom Extensions

Additional features beyond standard Markdown:

- Image sizing with `{.xs}`, `{.small}`, `{.medium}`, `{.large}`, `{.xl}`
- Alignment classes: `{.left}`, `{.center}`, `{.right}`
- Heading attributes: `{.notoc}`, `{.nonumber}`
- Figure groups with `:::figures` blocks
- Custom column widths: `{col-widths="20%,30%,50%"}`

B. Configuration Options

B.1 Document Metadata

Controlled via `config.yaml`:

1. **Document properties**: title, author, date, version
2. **Page layout**: margins, paper size, orientation
3. **Headers and footers**: content, alignment, dynamic fields

B.1.1 Dynamic Fields

Headers and footers support placeholder substitution:

- `{title}` - Document title from config

- `{author}` - Author name from config
- `{date}` - Document date from config
- `{page}` - Current page number
- `{total}` - Total page count

B.2 Styling Options

Visual presentation controlled through:

- **Heading colors**: Alternating dark blue and medium blue
- **Numbered headings**: Optional sequential numbering
- **Code formatting**: Grey background with monospace font
- **Table styling**: Alternating row shading, header formatting

C. Property Substitution

Dynamic values can be inserted using `{{property}}` syntax from `properties.yaml`. This enables template-based document generation where specific values are populated at build time.

C.1 Usage Examples

The converter supports **6 levels** heading levels and **3 styles** table styles. The **md_to_docx** project is currently at version **2.0** and has processed over **1,250+** files.

C.2 Configuration Data

Based on the configuration:

- **Supported formats**: Markdown to Word (.docx)
- **Parser engine**: Marko with GFM extensions
- **Maximum heading depth**: 6 levels
- **Available image sizes**: 5 classes (xs, small, medium, large, xl)
- **Alert types**: 4 types (note, tip, warning, caution)
- **Code language support**: Python, JavaScript, and any code

These values demonstrate how properties from YAML files are dynamically inserted into the document content at conversion time.

End of Document

10. Introduction

This document serves as a comprehensive demonstration of the md_to_docx converter capabilities. It showcases the various formatting features and document elements that can be automatically converted from Markdown syntax to professional Microsoft Word documents.

10.1 Purpose and Scope

The primary objectives of this showcase are to:

1. **Demonstrate document structure capabilities** including multi-level headings, automatic table of contents generation, and section organization
2. **Present text formatting features** such as emphasis, inline code, and paragraph styling
3. **Illustrate list functionality** including ordered, unordered, and nested list structures
4. **Exhibit table generation** with various formatting options and column width controls
5. **Showcase image handling** including standalone images, figure groups, and size controls
6. **Highlight cross-referencing** capabilities for figures, tables, and sections

11. Document Structure

11.1 Heading Hierarchy

The converter supports six levels of headings, automatically numbered and styled with a professional color scheme:

11.1.1 Level 3 Heading

This demonstrates the third level of the document hierarchy.

11.1.1.1 Level 4 Heading

Fourth level headings provide additional granularity for complex documents.

11.1.1.1.1 Level 5 Heading

Fifth level headings for detailed subsections.

11.1.1.1.1.1 Level 6 Heading

The deepest level of heading hierarchy available.

11.2 Heading Customization

Headings can be excluded from the table of contents using the `{.notoc}` attribute, or numbering can be suppressed with `{.nonumber}`.

12. Typography and Text Formatting

12.1 Basic Text Styles

The converter supports standard Markdown text formatting:

Italic text is used for emphasis or citations.

Bold text draws attention to important information.

Bold and italic combined creates strong emphasis.

~~Strikethrough~~ indicates removed or deprecated content.

12.2 Inline Code

Technical terms and code snippets can be displayed inline using backticks: `function_name()`, `variable_value`, or `CSS_property`.

12.3 Blockquotes

> *This is a standard blockquote. It can be used for important notes, citations, or highlighted information that stands out from the main text flow.*

Blockquotes feature a distinctive left border and italicized text for visual differentiation.

13. Lists and Enumeration

13.1 Unordered Lists

Simple bullet points for itemized information:

- First item in the list
- Second item with additional context
- Third item demonstrating the format

13.2 Ordered Lists

Numbered sequences for procedural documentation:

1. First step in the process
2. Second step with specific details
3. Third step completing the sequence

13.3 Nested Lists

Lists can be nested to show hierarchical relationships:

1. **Main category A**
    1. Sub-item A.1
    2. Sub-item A.2
        1. Detail A.2.1
        2. Detail A.2.2
    3. Sub-item A.3
2. **Main category B**
    1. Sub-item B.1
    2. Sub-item B.2
3. **Main category C**
    1. Sub-item C.1

This demonstrates the converter's ability to handle complex list structures with proper indentation and formatting.

14. Tables

14.1 Standard Data Table

Tables support headers, alternating row shading, and full-width formatting:

Feature	Status	Priority	Notes
Heading Numbering	Implemented	High	Configurable via YAML
Cross-references	Implemented	High	Bold + Italic style
Image Resizing	Implemented	Medium	Size classes supported
Table Styling	Implemented	High	Auto-formatting applied
Code Blocks	Implemented	Medium	Syntax highlighting

Table 4: *Feature implementation status and priorities.*

As shown in Table 4, all major features have been successfully implemented with appropriate priority levels assigned.

14.2 Table Without Header

Simple tables without headers are also supported:

Row 1, Cell 1	Row 1, Cell 2	Row 1, Cell 3	Row 1, Cell 4
Row 2, Cell 1	Row 2, Cell 2	Row 2, Cell 3	Row 2, Cell 4
Row 3, Cell 1	Row 3, Cell 2	Row 3, Cell 3	Row 3, Cell 4

Table 9: *Data visualization example without headers.*

14.3 Wide Table with Text Wrapping

Tables automatically handle text wrapping for content that exceeds column widths:

Component	Description	Implementation Details	Status
Parser	Marko library with GFM extensions	Handles all standard Markdown plus GitHub-style alerts and tables	Active

Builder	Document assembly engine	Manages section breaks, page numbering, and TOC generation	Active
Styles	Custom style definitions	Heading hierarchy, code blocks, captions, and alerts	Active
Images	Processing pipeline	Size calculation, aspect ratio preservation, placement	Active

Table 5: *System architecture components with detailed descriptions.*

Refer to Table 5 for a complete overview of the system architecture.

14.4 Table with Custom Column Widths

Tables can have custom column widths specified as percentages using the `{col-widths="..."}` directive placed immediately after the table:

Short	Medium	Long Description
ID	Category	Detailed explanation of the feature or component
001	Parser	Handles Markdown syntax parsing with GFM extensions
002	Builder	Assembles the document structure and applies formatting

Table 6: *Example table with custom column width distribution.*

In this example, the first column occupies 15% of the table width, the second column 25%, and the third column (containing longer descriptions) occupies 60% of the available space.

15. Images and Figures

15.1 Standalone Images

15.1.1 Full-Width Image

Large images can be displayed at full content width for maximum impact:

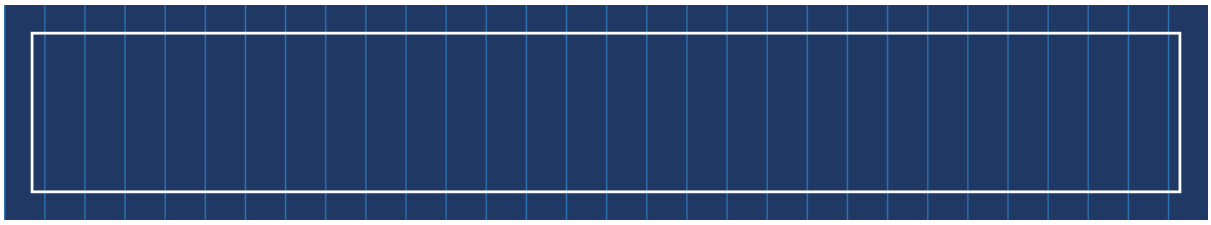


Figure 4: *Complete system overview diagram showing all major components and their relationships.*

As illustrated in Figure 4, the system architecture consists of four primary modules working in concert.

15.1.2 Left-Aligned Image

Images can be positioned on the left side with text flowing alongside:

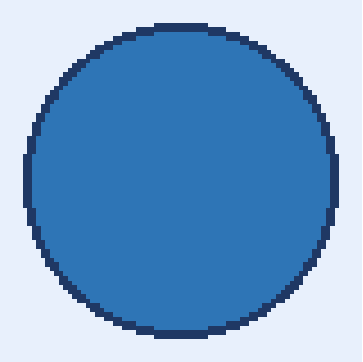


Figure 2: *Individual module icon representing the parsing engine.*

This layout demonstrates how content wraps around left-aligned images. The parsing module serves as the foundation of the conversion pipeline, responsible for transforming raw Markdown syntax into an abstract syntax tree (AST) that the builder can process.

15.1.3 Centered Medium Image

The default presentation centers images within the content area:

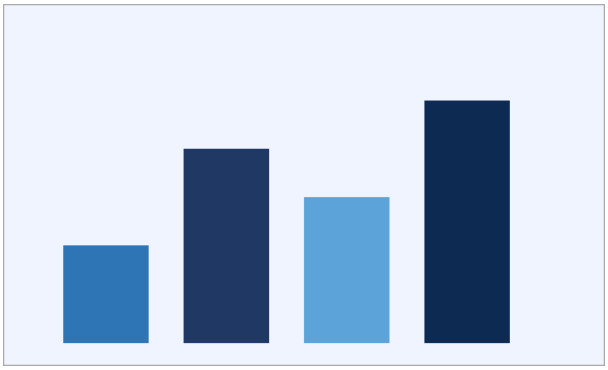
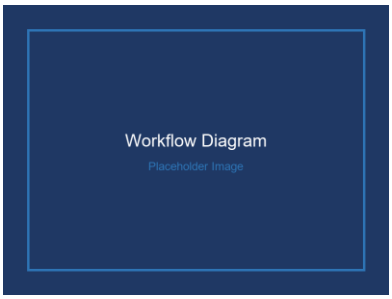


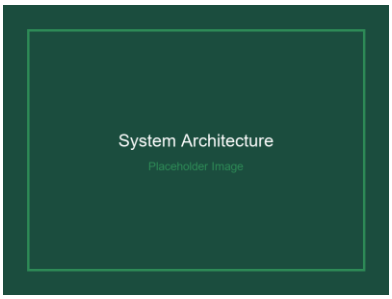
Figure 5: *Data flow diagram showing information pathways through the system.*

15.2 Figure Groups

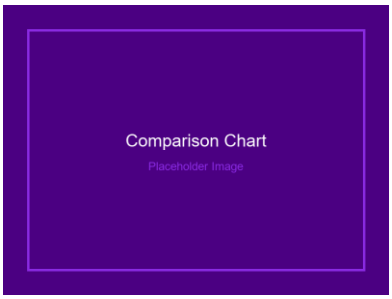
Multiple images can be displayed side-by-side for comparison purposes:



a) Input Processing



b) Output Generation



c) Quality Control

Figure 6: *Three-phase processing pipeline: input parsing, document building, and quality validation.*

The figure group above, referenced as Figure 6, demonstrates how multiple related images can be presented together with individual sub-captions (a, b, c) and an overall group caption.

16. Cross-References and Linking

16.1 Internal References

The document supports cross-referencing to:

Sections (e.g., Section 4.2 covers list functionality)

16.2 External Hyperlinks

External resources can be linked directly: Markdown Specification or Microsoft Word Documentation.

17. Code Examples

17.1 Python Code Block

```python
def process_markdown(source_file, config):
    """
    Main entry point for markdown processing.

    Args:
        source_file: Path to markdown input
        config: Dictionary of configuration options

    Returns:
        Document: Generated Word document object
    """
    builder = DocumentBuilder(config)
    builder.setup()

    with open(source_file, 'r') as f:
        content = f.read()

    builder.add_content(content)
    return builder.doc
```

Code Block 1: Python implementation of the main processing function.

17.2 JavaScript Code Block

```javascript
// Configuration parser for document settings
function parseConfig(configPath) {
  const fs = require('fs');
  const yaml = require('js-yaml');

  try {
    const config = yaml.load(fs.readFileSync(configPath, 'utf8'));
    return {
      margins: config.page || {},
      header: config.header || {},
      footer: config.footer || {},
      document: config.document || {}
    };
  } catch (error) {
    console.error('Configuration error:', error);
    return {};
  }
}
```

Code Block 2: JavaScript configuration parser implementation.

18. Special Elements

18.1 Alert Boxes

GitHub-style alerts provide visual emphasis for different message types:

> **[NOTE]** This is an informational note. It provides additional context or helpful information that supplements the main content.

> **[TIP]** This is a helpful tip. Tips offer practical advice or shortcuts that can improve workflow efficiency.

> **[WARNING]** This is a warning message. Warnings indicate potential issues or important considerations that require attention.

> **[CAUTION]** This is a caution alert. Cautions highlight actions that could cause problems if not handled carefully.

18.2 Horizontal Rules

Horizontal rules can be used to separate sections visually:

They create a clear break without starting a new page.

18.3 Vertical Space

The `:::space` directive provides precise vertical spacing control, ideal for fine-tuning layout on cover pages and between sections.

Use `lines` to insert space measured in body-text lines (11 pt per line):

Or use `pt` for an exact point value:

Both examples above create white space without forcing a page break.

Appendix

A. Supported Markdown Extensions

A.1 GitHub Flavored Markdown

The converter fully supports GFM syntax including:

- Tables with alignment
- Task lists (converted to bullet points)
- Strikethrough text
- Autolinks

18.3.1 A.1.1 Table Alignment

Tables support column alignment through colon placement:

Left	Center	Right
L1	C1	R1
L2	C2	R2

18.3.2 A.2 Custom Extensions

Additional features beyond standard Markdown:

- Image sizing with `{.xs}`, `{.small}`, `{.medium}`, `{.large}`, `{.xl}`
- Alignment classes: `{.left}`, `{.center}`, `{.right}`
- Heading attributes: `{.notoc}`, `{.nonumber}`
- Figure groups with `:::figures` blocks
- Custom column widths: `{col-widths="20%,30%,50%"}`

B. Configuration Options

B.1 Document Metadata

Controlled via `config.yaml`:

1. **Document properties**: title, author, date, version
2. **Page layout**: margins, paper size, orientation
3. **Headers and footers**: content, alignment, dynamic fields

B.1.1 Dynamic Fields

Headers and footers support placeholder substitution:

- `{title}` - Document title from config

- `{author}` - Author name from config
- `{date}` - Document date from config
- `{page}` - Current page number
- `{total}` - Total page count

B.2 Styling Options

Visual presentation controlled through:

- **Heading colors**: Alternating dark blue and medium blue
- **Numbered headings**: Optional sequential numbering
- **Code formatting**: Grey background with monospace font
- **Table styling**: Alternating row shading, header formatting

C. Property Substitution

Dynamic values can be inserted using `{{property}}` syntax from `properties.yaml`. This enables template-based document generation where specific values are populated at build time.

C.1 Usage Examples

The converter supports **6 levels** heading levels and **3 styles** table styles. The **md_to_docx** project is currently at version **2.0** and has processed over **1,250+** files.

C.2 Configuration Data

Based on the configuration:

- **Supported formats**: Markdown to Word (.docx)
- **Parser engine**: Marko with GFM extensions
- **Maximum heading depth**: 6 levels
- **Available image sizes**: 5 classes (xs, small, medium, large, xl)
- **Alert types**: 4 types (note, tip, warning, caution)
- **Code language support**: Python, JavaScript, and any code

These values demonstrate how properties from YAML files are dynamically inserted into the document content at conversion time.

End of Document